🔥 1⃣ University Payroll Management (Multilevel Inheritance)

📌 Question:

Create class Person:

- id, name

Create class Employee extends Person:

- basicSalary

Create class TeachingStaff extends Employee:

- subject, researchAllowance

Override method calculateSalary() in TeachingStaff:

- Gross Salary = basic + 20% HRA + 10% DA + researchAllowance

Store 5 employees (mix TeachingStaff & normal Employee) in array of Employee type and display payroll report.

🔥 2⃣ Smart Banking System (Abstract + Transaction Logic)

📌 Question:

Create abstract class Account:

- accountNumber
- holderName
- balance
- abstract method calculateInterest()

Create:

- SavingsAccount (4% interest)
- CurrentAccount (no interest, but ₹500 penalty if balance < 1000)

Implement:

- deposit()
- withdraw()

Store 4 accounts and simulate transactions.

🔥 3⃣ Advanced E-Commerce Platform (Interface + Polymorphism)

📌 Question:

Create interface Taxable:

- calculateTax()

Create class Product:

- id, name, price

Create subclasses:

- Electronics (18% GST)
- Clothing (5% GST)

Implement Taxable interface.

Store different products in array and calculate final bill including tax.

🔥 4⃣ Transport Reservation System (Abstract + Override + Array)

📌 Question:

Create abstract class Transport:

- route
- distance
- abstract calculateFare()

Create:

- Bus (₹5 per km)
- Train (₹3 per km + ₹50 reservation charge)
- Flight (₹10 per km + 18% tax)

Store transports in array and calculate total revenue.

🔥 5⃣ Hospital Billing System (Multiple Conditions)

📌 Question:

Create class Patient:

- id, name, baseCharge

Create subclasses:

- GeneralWard (₹1000/day)
- ICU (₹5000/day + 12% service tax)

Calculate total bill based on number of days.

Store 5 patients and display total hospital income.

🔥 6 Library Digital & Physical System (Abstract + Late Fine + Override)

📌 Question:

Create abstract class LibraryItem:

- id, title
- abstract calculateFine(daysLate)

Create:

- PrintedBook (₹5 per day)
- EBook (₹2 per day)
- Journal (₹10 per day + ₹50 fixed charge)

Store 6 items and calculate total fine.

🔥 7⃣ Online Course Portal (Interface + Abstract Combination)

📌 Question:

Create interface Certification:

- issueCertificate()

Create abstract class Course:

- courseCode
- fee
- abstract calculateTotalFee()

Create:

- OfflineCourse
- OnlineCourse (add platform fee)

Implement Certification in both classes.

Store 4 courses and display total income.

🔥 8⃣ Vehicle Insurance System (Method Overriding + Risk Logic)

📌 Question:

Create class Vehicle:

- vehicleNumber
- vehiclePrice
- calculateInsurance()

Create:

- Car (5% insurance)
- Bike (3% insurance)
- Truck (8% insurance + ₹2000 risk charge)

Store vehicles and calculate total premium.

🔥 9 Employee Performance Appraisal System

📌 Question:

Create class Employee:

- id, name, rating (1–5)

Create subclasses:

- HR
- Developer

Override method calculateBonus():

- Rating ≥4 → 20% bonus

- Rating 3 → 10%
- Below 3 → No bonus

Store 5 employees and generate appraisal report.

🔥 🔟 Multi-Level Banking Loan Approval System

📌 Question:

Create abstract class Loan:

- applicantName
- loanAmount
- abstract calculateInterest()

Create:

- PersonalLoan (12%)
- EducationLoan (8%)
- BusinessLoan (15%)

If loanAmount > 5,00,000 → add 2% risk interest.

Store 5 loans and display total payable amount.

🔥 11 Bank ATM System with Custom Exception

📌 Detailed Question:

Design a banking system with the following requirements:

1. Create a class Account with:

- accountNumber
- holderName
- balance

2. Provide:

- Constructor to initialize values
- Method deposit(double amount)
- Method withdraw(double amount)

3. If withdrawal amount is greater than balance, throw a user-defined exception:

- InsufficientBalanceException

4. Create 5 account objects using an array.
5. Perform deposit and withdrawal operations.
6. Handle exception using try-catch-finally.

🎯 Marks Distribution:

- Class + Constructor → 5
- Custom Exception → 5
- Array of Objects → 5
- Logic + Validation → 5
- Output + Exception Handling → 5

🔥 12 Student Admission System with Multiple Exceptions

📌 Detailed Question:

Create a class Student with:

- id
- name
- age
- marks

Rules:

- If age < 17 → throw InvalidAgeException
- If marks < 0 or > 100 → throw InvalidMarksException

Requirements:

- Use constructor validation
- Store 5 students in array
- Handle multiple catch blocks

🎯 Examiner Focus:

✓ Multiple user-defined exceptions

✓ Constructor validation

✓ Proper exception messages

🔥 13 Online Shopping System with Stock Validation

📌 Detailed Question:

Create class Product:

- productId
- name
- price
- stock

Create method:

- purchase(int quantity)

Rules:

- If quantity > stock → throw OutOfStockException
- After purchase, reduce stock

Store 5 products in array and simulate 3 purchase transactions.

🎯 What Examiner Checks:

✓ Business logic inside method

✓ Custom exception

✓ State update (stock reduction)

🔥 14 Library Fine Calculation with Abstract Class

📌 Detailed Question:

Create abstract class LibraryItem:

- id
- title
- abstract method calculateFine(int daysLate)

Create subclasses:

- Book → ₹5 per day
- Magazine → ₹3 per day

Rules:

- If daysLate < 0 → throw InvalidDaysException

Store items in array of parent type and calculate fine for 7 late days.

🎯 Concepts Covered:

✓ Abstract class

✓ Method overriding

✓ Runtime polymorphism

✓ Custom exception

🔥 15 Employee Payroll with Salary Validation

📌 Detailed Question:

Create class Employee:

- id
- name
- basicSalary

Rules:

- If salary < 0 → throw InvalidSalaryException

Create subclass Manager:

- Bonus = 20% of basic salary

Store 5 employees in array of Employee type and display gross salary.

🎯 Examiner Focus:

✓ Inheritance

✓ Overriding

✓ Exception inside constructor

🔥 16 Loan Approval System with Limit Exception

📌 Detailed Question:

Create abstract class Loan:

- applicantName
- loanAmount
- abstract calculateInterest()

Rules:

- If loanAmount > 10,00,000 → throw LoanLimitExceededException

Create:

- HomeLoan (8%)
- CarLoan (10%)

Calculate total payable = loanAmount + interest.

Store 5 loans and display report.

🔥 17 Electricity Billing with Unit Validation

📌 Detailed Question:

Create class ElectricityBill:

- consumerId
- units

Rules:

- If units < 0 → throw InvalidUnitException

Billing Slab:

- 0–100 → ₹5/unit
- 101–200 → ₹7/unit
- Above 200 → ₹10/unit

Store 5 consumers and display total revenue.

🔥 18 Online Examination Result System

📌 Detailed Question:

Create class Exam:

- studentName
- marks

Rules:

- If marks < 0 or > 100 → throw InvalidMarksException

Method:

- calculateResult()
- ≥ 40 → Pass

- < 40 → Fail

Store 5 students in array and display result.

🔥 19 Vehicle Insurance Premium System

📌 Detailed Question:

Create class Vehicle:

- vehicleNumber
- vehiclePrice

Rules:

- If price < 0 → throw InvalidPriceException

Create subclasses:

- Car → 5%
- Bike → 3%

Calculate premium and display total insurance collection.

🔥 20 Hotel Booking System with Availability Exception

📌 Detailed Question:

Create class Room:

- roomNumber
- isAvailable

Method:

- bookRoom()

Rules:

- If already booked → throw RoomNotAvailableException

Store 5 rooms in array and simulate booking process.

🔥 21Smart University Management System

📌 Detailed Question

Design a university management system with the following requirements:

Step 1: Create an Interface

Create an interface Payable:

- Method: double calculatePayment();

Step 2: Create an Abstract Class

Create an abstract class Person:

- id

- name
- Constructor to initialize values
- Abstract method displayDetails()

Step 3: Create Subclasses

Create:

- Student
- courseFee
- scholarship (deducted from fee)
- Faculty
- basicSalary
- bonus

Both must:

- Extend Person
- Implement Payable
- Override abstract methods

Step 4: Exception Handling

Create custom exception:

- InvalidAmountException (if fee or salary is negative)

Step 5:

- Store 5 mixed objects (Student + Faculty) in array of type Person
- Use polymorphism to calculate payments
- Handle exceptions using try-catch-finally

🎯 What Examiner Checks

✓ Constructor chaining

✓ Abstract method overriding

✓ Interface implementation

✓ Runtime polymorphism

✓ Custom exception

✓ Mixed object array

🔥 22 Advanced Banking & Insurance System

📌 Detailed Question

Step 1: Interface

Create interface Insurable:

- double calculateInsurance();

Step 2: Abstract Class

Create abstract class Account:

- accountNumber
- holderName
- balance

- Constructor
- Abstract method calculateInterest()

Step 3: Subclasses

Create:

- SavingsAccount
- CurrentAccount

Both:

- Extend Account
- Implement Insurable

Step 4: Business Rules

- If balance < 0 → throw InvalidBalanceException
- Savings → 4% interest
- Current → no interest
- Insurance = 2% of balance

Step 5:

Store accounts in array and generate full financial report.

🔥 23 Smart Transport Booking System

📌 Detailed Question

Step 1: Interface

Create interface Bookable:

- void bookSeat(int seats)

Step 2: Abstract Class

Create abstract class Transport:

- route
- baseFare
- Constructor
- Abstract method calculateFare(int distance)

Step 3: Subclasses

Create:

- Bus
- Train
- Flight

Rules:

- Flight adds 18% GST
- Train adds ₹50 reservation charge

Step 4: Exception

If seats requested > availableSeats → throw SeatNotAvailableException

Step 5:

Store 5 transports in array and simulate booking.

🔥 24 E-Commerce Billing & Taxation System

📌 Detailed Question

Step 1: Interface

Create interface Taxable:

- double calculateTax();

Step 2: Abstract Class

Create abstract class Product:

- id
- name
- price
- Constructor
- Abstract method double calculateFinalPrice();

Step 3: Subclasses

Create:

- Electronics
- Grocery
- Clothing

Tax Rules:

- Electronics → 18%
- Grocery → 5%
- Clothing → 12%

Step 4: Exception

If price < 0 → throw InvalidPriceException

Step 5:

Store products in array and calculate total bill.

🔥 25 Hospital Billing & Insurance System

📌 Detailed Question

Step 1: Interface

Create interface Insurable:

- double calculateInsuranceCoverage();

Step 2: Abstract Class

Create abstract class Patient:

- id

- name
- baseCharge
- Constructor
- Abstract method double calculateBill(int days);

Step 3: Subclasses

Create:

- GeneralWard
- ICU
- Emergency

Rules:

- ICU adds 12% service tax
- Emergency adds ₹2000 emergency charge

Step 4: Exception

If days <= 0 → throw InvalidStayException

Step 5:

Store patients in array and display final payable amount after insurance deduction.

🔥 26 Online Examination & Certification System

📌 Detailed Question

Step 1: Interface

Create interface Certifiable:

- void issueCertificate();

Step 2: Abstract Class

Create abstract class Exam:

- subject
- marks
- Constructor
- Abstract method calculateResult()

Step 3: Subclasses

Create:

- MCQExam
- PracticalExam

Rules:

- Pass if marks ≥ 40
- If marks < 0 or > 100 → throw InvalidMarksException

Step 4:

Store exams in array and issue certificates to passed students.

🔥 27 Multi-Level Loan & Risk Assessment System

📌 Detailed Question

Step 1:

Create abstract class Loan:

- applicantName
- loanAmount
- Constructor
- Abstract method calculateInterest()

Step 2:

Create interface RiskAssessment:

- double calculateRisk();

Step 3:

Create:

- HomeLoan
- BusinessLoan

Rules:

- If loanAmount > 5,00,000 → add 2% risk
- If loanAmount negative → throw InvalidLoanAmountException

Step 4:

Store loans in array and calculate total payable amount.

___

28 Define a class student with the following specifications: Private members of the class: Admission Number - An Integer Name - string of 20 characters Class - Integer Roll Number - Integer Public members of the class: 146 getdata() - To input the data showdata() - To display the data Write a program to define an array of 10 objects of this class, input the data in this array and then display this list.

29 Create a class Customer with properties customer ID, name, address, phone number, date of birth and function to get and print these attributes. Inherit the class Account from Customer class with account number, account type, rate of interest and balance properties and functions to get and print these properties. Also in account class implement the deposit and withdraw function. Use appropriate access modifier with attributes and methods of each class.

30

For educational institute design an application in which a Person with person id, name, address, department and phone number can be a faculty or a student. A faculty can have other attributes like degree, designation, specialization, experience etc. A student can have attributes like semester; results etc. create appropriate classes and methods in the classes. Use 166 appropriate access modifier with attributes and methods of each class. Also show their use in main method.

31 Write a program with an array that is initialized with test data. Use any primitive data type of your choice. The program should also have the following methods:

- gettotal. This meth d should accept a one-dimensional array as its argument and return the total of the values in the array.
- getAverage. This method should accept a one-dimensional array as its argument

- gethighest. This method should accept a one-dimensional array as its argume and return the highest value in the array.
- getLowest. This method should accept a one-dimensional array as its argume and return the lowest value in the array.

Demonstrate each of the methods in the program.

32 Month Class

Write a class named Month. The class should have an int field named monthiamber that holds the number of the month. For example, January would be 1, February would be 2. and so forth. In addition, provide the following methods:

- Ano-arg constructor that sets the monthumber held to 1.
- A constructor that accepts the number of the month as an argument. It should set the monthNumber field to the value passed as the argument. If a value less than 1 or greater than 12 is passed, the constructor should set monthumber to 1.
-
- Chapter 9A Second Look at Classes and Objects
- • A constructor that accepts the name of the month, such as "January" or
- "February" as an argument. It should set the monthNumber field to the correct corresponding value.
- A setMonthNumber method that accepts an int argument, which is assigned to the monthNumber field. If a value less than 1 or greater than 12 is passed, the method should set monthNumber to 1.
- A getMonthNumber method that returns the value in the monthNumber field.
- A getMonthName method that returns the name of the month. For example, if the monthNumber field contains 1, then this method should return "January".
- A tostring method that returns the same value as the getMonthName method.

- A greaterThan method that accepts a Month object as an argument. If the calling object's monthNumber field is greater than the argument's monthNumber field, this method should return true. Otherwise, it should return false.
- A lessThan method that accepts a Month object as an argument. If the calling object's monthNumber field is less than the argument's monthNumber field, this method should return true. Otherwise, it should return false.

33 Word Counter

Write a method that accepts a string object as an argument and returns the number of wards it contains. For instance, if the argument is "Four score and seven years ago" the method should return thebnumber 6. Demonstrate the method in a program that asks the should be displayed on the screen.

user to input a string and then passes it to the method. The number of words in the string

Programming Challenges

34 Write a Java program with an interface Printable having a method printDetails(). Create two classes Document and Photo that implement this interface. Implement the method in each to print specific details, and demonstrate polymorphism by calling the method on objects of both classes.

35 Write a Java program for an abstract class Employee with attributes name and salary, and an abstract method calculateBonus(). Create two subclasses Manager and Developer that implement the bonus calculation differently (e.g., 20% for Manager, 10% for Developer). Create objects and display name, salary, and bonus for each.

36An educational institution wishes to maintain a database of its employees. The database is divided into a number of classes whose hierarchical relationships are as follows: Staff (code, name): the base class Teacher (subject, publication):child class of the Staff Officer (grade): child class of the Staff. Note that the information given in brackets specifies the minimum information required for each class. Specify all classes and define functions and constructors to create the database and retrieve information as and when needed.